Guía completa de Machine Learning: regresión lineal, polinómica y regularización

Desde cero, con rigor,
para entender, defender y aplicar

Bootcamp The Bridge · Bloque de Machine Learning

Material de estudio — todos los conceptos explicados paso a paso

Índice

Esta guía cubre todo el contenido de los cuatro bloques de Machine Learning que vimos (y que te perdiste) en el bootcamp. Cada concepto se explica primero con palabras corrientes, después con la fórmula matemática, y por último con un ejemplo de código Python para que veas cómo se usa en sklearn. Lee las secciones en orden: cada una se apoya en lo anterior.

Parte I — Fundamentos de Machine Learning

1. Qué es el Machine Learning

2. Regresión lineal: la idea base

2.1 Regresión lineal simple

2.2 Regresión lineal múltiple

2.3 Interpretación de los coeficientes β

3. Cómo encuentra el modelo la "mejor recta": función de costes y Gradient Descent

3.1 La función de costes (MSE)

3.2 Gradient Descent (descenso del gradiente)

3.3 Learning rate

Parte II — Medir el error

4. Residuos: la base de todas las métricas

5. MAE — Error Absoluto Medio

6. MSE — Error Cuadrático Medio

7. RMSE — Raíz del Error Cuadrático Medio

8. MAPE — Error Porcentual Absoluto Medio

9. R^2 — Coeficiente de determinación

10. Correlación de Pearson y p-valor

Parte III — Metodología correcta

11. Train / Test split

12. Overfitting, Underfitting y el dilema sesgo-varianza

13. StandardScaler y MinMaxScaler

14. Validación cruzada (cross_val_score)

15. Condiciones para aplicar regresión lineal

15.1 Linealidad

15.2 Multicolinealidad

15.3 Homocedasticidad

15.4 Independencia y normalidad de los residuos

Parte IV — Regresión polinómica

16. Cuando la relación no es lineal

17. PolynomialFeatures de sklearn

18. Elección del grado óptimo

Parte V — Regularización

19. Por qué regularizar

20. Ridge (L2)

21. Lasso (L1)

22. Elastic Net

23. Cómo elegir entre Ridge, Lasso y Elastic Net

Parte VI — Cheat sheet y glosario

24. Pipeline típico de un problema de regresión

25. Cheat sheet de métricas y modelos

26. Glosario en orden alfabético

Parte I
Fundamentos de Machine Learning

1. Qué es el Machine Learning

Antes de meternos con regresiones, conviene saber dónde estamos. El **Machine Learning** (aprendizaje automático) es una rama de la inteligencia artificial en la que un ordenador aprende patrones a partir de ejemplos, sin que nadie le programe reglas explícitas. En lugar de decirle «si el precio sube, haz esto», le enseñas miles de casos pasados y deja que él solo descubra la relación.

Dentro del Machine Learning hay tres grandes familias:

- **Aprendizaje supervisado**. Tienes datos *etiquetados*: para cada ejemplo conoces la respuesta correcta. El modelo aprende a predecirla. Este es nuestro terreno.
- **Aprendizaje no supervisado**. Tienes datos sin etiquetar y el modelo busca agrupaciones o estructuras (clustering, reducción de dimensionalidad, etc.).
- **Aprendizaje por refuerzo**. Un agente aprende por ensayo y error recibiendo recompensas (videojuegos, robótica).

Dentro del supervisado hay dos tipos de problema según lo que quieras predecir:

- **Regresión**. La respuesta es un número continuo: el precio de una casa, la temperatura mañana, las ventas del próximo mes. Todo lo que vas a ver en esta guía.
- **Clasificación**. La respuesta es una categoría: spam o no spam, perro o gato, diabético o no. Se verá en otro bloque del bootcamp.

> **QUEDÉMONOS CON ESTO**
>
> El Machine Learning supervisado de regresión consiste en: *tengo ejemplos pasados con la respuesta correcta (entrenamiento); el modelo aprende la relación entre las variables; después uso esa relación para predecir respuestas sobre datos nuevos (test).*

2. Regresión lineal: la idea base

2.1 Regresión lineal simple

Imagínate una nube de puntos en un gráfico. En el eje X pones una variable (por ejemplo, las horas que ha estudiado cada alumno) y en el eje Y otra (la nota que ha sacado). Cada alumno es un punto.

La **regresión lineal simple** consiste en dibujar la mejor línea recta posible por esa nube. ¿Para qué? Para que, cuando aparezca un alumno nuevo y te diga cuántas horas ha estudiado, puedas usar la recta para predecir su nota.

La ecuación de una recta es:

$$Y = \beta_0 + \beta_1 \cdot X$$

- **Y** es lo que queremos predecir (la nota). Se llama *variable dependiente*, *target*, *respuesta* o *regresando*.
- **X** es lo que conocemos (las horas de estudio). Se llama *variable independiente*, *feature*, *predictor* o *regresor*.
- $\boldsymbol{\beta_0}$ (se lee «beta sub cero») es el **intercepto** u **ordenada en el origen**. Es el valor de Y cuando X vale 0: el punto por donde la recta cruza el eje vertical.
- $\boldsymbol{\beta_1}$ es la **pendiente** o **coeficiente**: cuánto sube o baja Y por cada unidad que aumenta X.

EJEMPLO NUMÉRICO

Supón que la recta aprendida es *nota = 2 + 0.8 · horas*. Eso significa:

- Un alumno que estudió 0 horas: la nota predicha sería 2 (el intercepto).
- Por cada hora extra, la nota sube 0.8 puntos.
- Un alumno que estudió 5 horas: *2 + 0.8 × 5 = 6*.

2.2 Regresión lineal múltiple

En la vida real rara vez predices algo a partir de una sola variable. El precio de una casa no depende solo de los metros cuadrados, también del barrio, el número de habitaciones, la antigüedad, etc. Cuando hay varias features, la ecuación se extiende:

$$Y = \beta_0 + \beta_1 \cdot X_1 + \beta_2 \cdot X_2 + \beta_3 \cdot X_3 + \ldots + \beta_j \cdot X_j$$

Cada feature tiene su propio coeficiente β. En la diapositiva que vio tu clase, el ejemplo era:

$$\text{Sales} = 500 + 50 \cdot \text{TV} + 20 \cdot \text{radio} + 15 \cdot \text{newspaper}$$

Geométricamente, en vez de una recta tenemos un **hiperplano** en un espacio de tantas dimensiones como features. No lo podemos dibujar si hay más de 2 features, pero la matemática es la misma.

2.3 Interpretación de los coeficientes β

Ésta es una parte que te van a preguntar seguro en cualquier entrevista o examen. Los coeficientes β no son cualquier cosa: **tienen significado real**.

> **LA REGLA DE ORO PARA INTERPRETAR B_J**
>
> β_j es *cuánto varía Y cuando la feature X_j aumenta en 1 unidad, manteniendo el resto de features constantes.*
>
> Si β_j es positivo, cuando X_j sube, Y sube. Si es negativo, cuando X_j sube, Y baja.

Aplicado al ejemplo de ventas:

- β_{TV} = 50: por cada euro extra invertido en TV (manteniendo radio y prensa igual), las ventas suben 50.
- β_{radio} = 20: por cada euro extra en radio, las ventas suben 20.
- $\beta_{newspaper}$ = 15: por cada euro extra en prensa, las ventas suben 15.

β_0 = 500 sería lo que se vendería si no invirtiéramos nada en publicidad. A menudo el intercepto no tiene una interpretación útil por sí solo (a veces esas condiciones son absurdas, como «una casa de 0 m²»), pero matemáticamente existe.

> **OJO: LOS B SIN ESCALAR NO REFLEJAN IMPORTANCIA**
>
> Podrías pensar: «β_{TV} = 50 es mayor que β_{radio} = 20, luego TV es más importante». Eso es un error típico. Los coeficientes están en las unidades de cada variable, y comparar coeficientes de variables en escalas distintas no tiene sentido (por ejemplo, comparar euros con número de habitaciones). Para comparar la importancia hay que estandarizar los datos primero — lo vemos en la sección del *StandardScaler*.

3. Cómo encuentra el modelo la "mejor recta": función de costes y Gradient Descent

Hasta ahora hemos hablado del modelo como si la recta apareciese por arte de magia. Pero el ordenador tiene que calcular los β óptimos de algún modo. Este apartado es el que tu profesor dio en clase y que te perdiste — es la mecánica interna del aprendizaje.

3.1 La función de costes (MSE)

Por una nube de puntos pueden pasar infinitas rectas. ¿Cuál es la «mejor»? Necesitamos un criterio numérico que diga qué recta se ajusta mejor a los datos. Ese criterio se llama **función de costes** o *loss function*.

Para la regresión lineal, la función de costes estándar es el **Error Cuadrático Medio** (MSE, *Mean Squared Error*). Se define como la media de los errores al cuadrado:

$$\mathrm{MSE} = (1/\mathrm{N}) \cdot \Sigma\ (\mathrm{Y_i} - \hat{\mathrm{Y}}_\mathrm{i})^2$$

Donde:

- **N** es el número de observaciones.
- $\mathbf{Y_i}$ es el valor real del i-ésimo ejemplo.
- $\hat{\mathbf{Y}}_\mathbf{i}$ (léase «Y sombrero sub i») es el valor predicho por el modelo para ese ejemplo.
- $\mathbf{(Y_i - \hat{Y}_i)}$ es el error o residuo de ese ejemplo (en el capítulo 4 profundizamos).

> **POR QUÉ AL CUADRADO**
>
> Si sumásemos los errores tal cual, los positivos y los negativos se cancelarían entre sí y el total daría casi cero aunque la recta fuera horrible. Al elevar al cuadrado conseguimos dos cosas: todos los errores se vuelven positivos, y los errores grandes pesan mucho más que los pequeños (un error de 4 aporta 16, uno de 2 aporta solo 4). Esto es deseable porque normalmente queremos evitar equivocaciones grandes a toda costa.

Así que nuestro objetivo matemático es claro: **encontrar los valores de β_0, β_1, ..., β_j que minimizan la función de costes MSE**. Es un problema de optimización.

3.2 Gradient Descent (descenso del gradiente)

Para problemas simples podríamos calcular los β óptimos directamente con una fórmula cerrada (*Ordinary Least Squares*, OLS). Pero cuando hay muchas features o modelos más complejos, no es práctico. Usamos un método iterativo llamado **Gradient Descent**.

> **LA METÁFORA DE LA MONTAÑA CON NIEBLA**
>
> Imagínate que estás en una montaña rodeado de niebla espesa y quieres llegar al valle. No ves a lo lejos. ¿Qué haces? Miras la pendiente del suelo justo donde pisas, y das un paso en la dirección más descendente. Repites. Paso a paso, sin ver lejos, vas bajando hasta que el suelo deja de descender: has llegado al valle. El Gradient Descent hace exactamente eso con la función de costes.

Matemáticamente, la «pendiente del suelo» se calcula con el **gradiente**, que es la derivada parcial de la función de costes respecto a cada parámetro β. El gradiente te dice hacia dónde crece la función; nosotros queremos lo contrario, así que caminamos en dirección opuesta (de ahí «descenso»).

El algoritmo es así:

1. Empezamos con valores aleatorios de β.
2. Calculamos la función de costes (MSE) con esos β.
3. Calculamos el gradiente (las derivadas parciales de la función de costes respecto a cada β).
4. Actualizamos cada β restando una pequeña fracción del gradiente: $\beta_{nuevo} = \beta_{viejo} - \alpha \cdot \text{gradiente}$.
5. Volvemos al paso 2. Iteramos hasta que el MSE deja de bajar (convergencia).

La intuición visual con una curva $f(x) = x^2$ es muy útil:

- Si estoy a la derecha del mínimo, la pendiente es positiva → tengo que ir hacia la izquierda (restar del X actual).
- Si estoy a la izquierda del mínimo, la pendiente es negativa → tengo que ir hacia la derecha (sumar al X actual).
- Si la pendiente es cero, estoy en el mínimo → he convergido.

El truco es que al restar α · gradiente, el signo ya se encarga automáticamente: si el gradiente es positivo, resto; si es negativo, al restar un número negativo sumo.

3.3 Learning rate (α)

El parámetro α (alfa) que aparece en la fórmula $\beta_{nuevo} = \beta_{viejo} - \alpha \cdot gradiente$ se llama **learning rate** o tasa de aprendizaje. Controla el tamaño de cada paso.

- Si α es **demasiado pequeño**: avanzas a pasitos minúsculos. Tardas muchísimo en llegar al mínimo (miles de iteraciones).
- Si α es **demasiado grande**: das zancadas gigantes. Puedes pasarte de largo del mínimo y, en el peor de los casos, oscilar sin converger nunca, o incluso divergir y mandar el modelo al infinito.
- Si α es **apropiado**: avanzas decididamente al principio y vas afinando cerca del mínimo.

El learning rate es un **hiperparámetro**: lo decides tú, no lo aprende el modelo. Valores típicos van de 0.001 a 0.1. En la regresión lineal de sklearn esto te da igual porque sklearn resuelve directamente con la fórmula cerrada, pero en otros modelos (redes neuronales) el learning rate es crítico.

VOCABULARIO A RETENER

Función de costes (loss function)

Métrica que cuantifica el error del modelo. Para regresión lineal, el MSE.

Gradient Descent

Algoritmo iterativo que busca los parámetros que minimizan la función de costes.

Gradiente

Vector de derivadas parciales. Indica la dirección en la que crece más rápido la función.

Learning rate (α)

Tamaño del paso en cada iteración del Gradient Descent.

Convergencia

Cuando el algoritmo deja de mejorar: el modelo ha encontrado el mínimo (o se ha estancado).

Hiperparámetro

Configuración del algoritmo que decides tú antes de entrenar (learning rate, alpha de Ridge, grado del polinomio…). No lo aprende el modelo de los datos.

Parámetro

Valores que el modelo aprende durante el entrenamiento (los β en regresión lineal).

Parte II
Medir el error

Una vez entrenado el modelo, hay que saber si es bueno o malo. Para eso existen varias métricas. Cada una mide algo ligeramente distinto y tiene sus pros y sus contras. Esta parte es imprescindible: todo el vocabulario MAE, MSE, RMSE, MAPE, R^2 que aparece mil veces en los notebooks vive aquí.

4. Residuos: la base de todas las métricas

Cuando el modelo hace una predicción, casi nunca acierta exactamente el valor real. Hay una diferencia entre lo que predijo y lo que realmente pasó. Esa diferencia se llama **residuo** o **error**:

$$\text{error}_i = Y_i - \hat{Y}_i$$

Donde Y_i es el valor real y $\hat{Y}_i$ es la predicción del modelo para el ejemplo i.

- Si el residuo es **positivo**, el modelo ha *subestimado* (el real es mayor que lo predicho).
- Si es **negativo**, el modelo ha *sobreestimado* (ha predicho más de lo que era).
- Si es **cero**, ha acertado ese punto exactamente.

Cada ejemplo del conjunto de test tiene su propio residuo. El problema es resumir todos esos residuos en un único número que nos diga «cómo de bueno es mi modelo». De ahí nacen las métricas.

5. MAE — Error Absoluto Medio

> **QUÉ ES EN UNA FRASE**
>
> En promedio, ¿cuánto se equivoca mi modelo en cada predicción, sin importar si se pasa o se queda corto?

Fórmula:

$$\text{MAE} = (1/N) \cdot \Sigma\, |Y_i - \hat{Y}_i|$$

Tomamos el valor absoluto de cada error (para que todos sean positivos y no se cancelen entre sí), los sumamos y los dividimos por el número de ejemplos. El resultado está en las mismas unidades que el target.

EJEMPLO NUMÉRICO

Si mis errores son +3, −2, +4, −1:

1. Valores absolutos: 3, 2, 4, 1
2. Suma: 3 + 2 + 4 + 1 = 10
3. Media: 10 / 4 = **2.5**

MAE = 2.5. Interpretación: «en promedio, mi modelo se equivoca en 2.5 unidades en cada predicción».

Ventaja: es el más fácil de interpretar. Si predices precios en euros, el MAE te dice «me equivoco de media en X euros».

Inconveniente: trata a todos los errores por igual. Un error grande y uno pequeño pesan proporcionalmente. Si en tu aplicación los errores grandes son especialmente graves (por ejemplo, predicciones médicas), puede ser insuficiente.

Cómo se calcula en sklearn:

```
from sklearn.metrics import mean_absolute_error
mae = mean_absolute_error(y_test, y_pred)
```

6. MSE — Error Cuadrático Medio

QUÉ ES EN UNA FRASE

Como el MAE, pero elevamos los errores al cuadrado antes de promediar. Esto castiga mucho más los errores grandes.

Fórmula:

$$\mathrm{MSE} = (1/\mathrm{N}) \cdot \Sigma\ (\mathrm{Y}_{\mathrm{i}} - \hat{\mathrm{Y}}_{\mathrm{i}})^2$$

EJEMPLO NUMÉRICO

Con los mismos errores +3, −2, +4, −1:

1. Al cuadrado: 9, 4, 16, 1
2. Suma: 30
3. Media: 30 / 4 = **7.5**

MSE = 7.5. Fíjate en la diferencia: el error de 4 aporta 16 al MSE (frente a 4 al MAE). Eleva mucho el resultado.

Ventaja: castiga errores grandes. Útil cuando en tu problema esos errores son especialmente costosos.

Inconveniente grave: las unidades están al cuadrado. Si predices precios en euros, el MSE está en «euros al cuadrado», que no significa nada intuitivo. Por eso casi nunca se reporta solo: se reporta su raíz (el RMSE).

En sklearn:

```
from sklearn.metrics import mean_squared_error
mse = mean_squared_error(y_test, y_pred)
```

7. RMSE — Raíz del Error Cuadrático Medio

QUÉ ES EN UNA FRASE

El MSE al que le aplicamos la raíz cuadrada. Así recuperamos las unidades originales.

Fórmula:

$$\text{RMSE} = \sqrt{\text{MSE}}$$

Es lo mejor de los dos mundos: castiga errores grandes (como el MSE) pero es interpretable en las unidades del target (como el MAE). Por eso es la métrica más usada para reportar.

COMPARAR MAE CON RMSE ES DIAGNÓSTICO

El RMSE siempre es mayor o igual que el MAE. La diferencia entre ambos te chiva cómo están distribuidos los errores:

- Si MAE y RMSE son parecidos: los errores son uniformes, ninguno especialmente grande.
- Si RMSE es mucho mayor que MAE: hay unos pocos errores muy grandes (outliers) que están tirando del RMSE hacia arriba.

sklearn no tiene función `root_mean_squared_error` directa (depende de la versión). Lo típico es:

```
import numpy as np
from sklearn.metrics import mean_squared_error
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
```

8. MAPE — Error Porcentual Absoluto Medio

QUÉ ES EN UNA FRASE

El error expresado como porcentaje del valor real. Útil cuando el target varía mucho en escala y quieres saber el error relativo.

Fórmula:

$$\mathrm{MAPE} = (1/N) \cdot \Sigma \mid (Y_i - \hat{Y}_i) \;/\; Y_i \mid \times 100$$

EJEMPLO QUE ACLARA POR QUÉ EXISTE EL MAPE

Tenemos dos pacientes con el mismo error absoluto:

Paciente	Real	Predicho	Error abs	Error %
A	200	190	10	5%
B	50	40	10	20%

El error absoluto es el mismo (10), pero fallar 10 sobre un real de 200 no es tan grave como fallar 10 sobre un real de 50. El MAPE lo captura.

Ventaja: es intuitivo (todo el mundo entiende «me equivoco un 7%») y es independiente de la escala, útil para comparar modelos entre problemas distintos.

Inconvenientes importantes:

- Si algún Y real vale 0, la división explota (no se puede dividir entre cero).
- Si algún Y real es muy pequeño, el porcentaje de ese caso puede ser enorme y distorsionar la media.
- No es simétrico: penaliza más subestimar que sobreestimar.

En sklearn:

```
from sklearn.metrics import mean_absolute_percentage_error
mape = mean_absolute_percentage_error(y_test, y_pred) * 100  # multiplicamos
por 100 para expresarlo en %
```

Nota: sklearn devuelve una proporción (0.07 significa 7%), por eso multiplicamos por 100.

9. R^2 — Coeficiente de determinación

Esta es la métrica más reveladora. MAE, MSE, RMSE y MAPE te dicen *cuánto* te equivocas en términos absolutos o relativos. Pero el **R^2** te dice algo más profundo: *si tu modelo sirve para algo o no*.

La idea del baseline tonto

Imagina que no tienes ningún modelo. ¿Cuál es la predicción más tonta que podrías hacer? **Responder siempre la media**. Si la nota media de una clase es 4.2, pues dices 4.2 para todos, sin mirar nada. Es el baseline más simple y más pobre posible.

El R^2 compara tu modelo con ese baseline tonto:

Valor de R^2	Significa
$R^2 = 1$	Modelo perfecto. Acierta todo sin error.
$R^2 = 0.8$	El modelo explica el 80% de la variabilidad del target.
$R^2 = 0.5$	El modelo explica la mitad. Decente según contexto.
$R^2 = 0$	Tu modelo predice igual de bien que responder siempre la media. No aporta nada.
$R^2 < 0$ (negativo)	Tu modelo predice peor que responder siempre la media. Has hecho un modelo que perjudica.

Fórmula:

$$R^2 = 1 - (SS_{res} \,/\, SS_{tot})$$

Donde:

- **SS_{res}** (Residual Sum of Squares): $\Sigma(Y_i - \hat{Y}_i)^2$. La suma de los errores al cuadrado del modelo.
- **SS_{tot}** (Total Sum of Squares): $\Sigma(Y_i - \bar{y})^2$. La suma de los errores al cuadrado del baseline tonto (predecir siempre la media $\bar{y}$).

POR QUÉ LA FÓRMULA TIENE SENTIDO

SS_{res} / SS_{tot} es la fracción del error que NO has conseguido eliminar respecto al baseline. Si tu modelo elimina el 80% del error, $SS_{res}/SS_{tot} = 0.2$, y $R^2 = 1 - 0.2 = 0.8$. Si tu modelo es igual que el baseline, esa fracción es 1, y $R^2 = 0$. Si tu modelo es peor ($SS_{res} > SS_{tot}$), la fracción supera 1 y R^2 se vuelve negativo.

EL R^2 NEGATIVO ES UNA ALARMA ROJA

Si entrenas un modelo y el R^2 en test sale negativo, el modelo *no sirve para nada* — literalmente, lo habrías hecho mejor ignorando los datos y respondiendo siempre la media. Esto pasa típicamente cuando las features no tienen relación real con el target (véase el ejercicio de Height: edad no predice altura en adultos) o cuando el modelo está muy mal calibrado.

En sklearn se calcula de dos formas equivalentes:

```
# Opción 1: método score del modelo
r2 = modelo.score(X_test, y_test)

# Opción 2: función de métricas
from sklearn.metrics import r2_score
r2 = r2_score(y_test, y_pred)
```

10. Correlación de Pearson y p-valor

Estas dos herramientas no son métricas del modelo propiamente, sino del análisis exploratorio previo. Te ayudan a saber si tiene sentido intentar una regresión lineal o no.

10.1 Correlación de Pearson

La **correlación de Pearson** es un número entre −1 y +1 que te dice si dos variables tienden a moverse juntas de forma lineal.

Valor de r	Interpretación
r = +1	Correlación positiva perfecta: cuando X sube, Y sube en línea recta.
r ≈ +0.7	Correlación positiva fuerte (p. ej. altura y peso de adultos).
r ≈ +0.3	Correlación positiva débil.
r = 0	No hay relación lineal.
r ≈ −0.3	Correlación negativa débil.
r = −1	Correlación negativa perfecta: cuando X sube, Y baja en línea recta.

UN DATO MUY ÚTIL: R AL CUADRADO

Si elevas la correlación al cuadrado (r^2) obtienes el porcentaje de variabilidad de Y que explica X linealmente. Si r = 0.7, entonces $r^2 = 0.49$: la variable X explica el 49% de la variabilidad de Y.

Esta r^2 en un problema con una sola feature coincide con el R^2 del modelo de regresión lineal. ¡No es casualidad!

10.2 Limitaciones de Pearson

- Solo detecta relaciones **lineales**. Si dos variables tienen relación en forma de U o curva, Pearson puede dar casi cero aunque la relación exista (ahí entra la regresión polinómica).
- **Correlación no implica causalidad**. El ejemplo clásico: el número de cigüeñas y el número de nacimientos en un país correlacionan positivamente. Pero no es que las cigüeñas traigan bebés: ambas cosas correlacionan con variables rurales/urbanas. Siempre que veas correlación, pregúntate qué variable oculta puede estar detrás.

10.3 P-valor: ¿es fiable esta correlación?

Cuando tienes pocos datos, cualquier correlación que midas podría deberse al puro azar. Necesitas una herramienta que te diga si lo que ves es señal real o ruido. Esa herramienta es el **p-valor**.

QUÉ MIDE EL P-VALOR

El p-valor es la probabilidad de observar una correlación como la que tienes (o más extrema) *si en la realidad no hubiera ninguna relación entre X e Y*.

La convención es:

- **p-valor < 0.05:** poco probable que sea casualidad → la correlación se considera *estadísticamente significativa*.

- **p-valor ≥ 0.05:** podría ser casualidad perfectamente → *no puedes afirmar* que haya relación.

El umbral de 0.05 es una convención arbitraria, no una ley física. Algunos estudios usan 0.01 o 0.001 para ser más estrictos.

POR QUÉ DEPENDE DEL TAMAÑO DE LA MUESTRA

Con 20 observaciones, una correlación de 0.25 probablemente tenga $p \approx 0.3$ (no significativa). Con las mismas variables y 1000 observaciones, la misma correlación tendrá $p < 0.001$ (muy significativa). Más datos = más capacidad para detectar relaciones reales.

Cálculo con scipy:

Parte III
Metodología correcta

Entrenar un modelo no es solo hacer model.fit(). Si no sigues una metodología correcta, tu modelo puede parecer excelente en tus datos y fracasar estrepitosamente cuando lo pongas a predecir casos nuevos. Esta parte es la que separa a un data analyst profesional del que solo copia notebooks.

11. Train / Test split

11.1 El problema que resuelve

Si entreno mi modelo con todos los datos disponibles y después evalúo el error con esos mismos datos, estoy haciendo trampa. El modelo puede **memorizar** los datos en lugar de aprender patrones generales. En los datos de entrenamiento parecerá que acierta mucho, pero con datos nuevos fallará.

Este fenómeno se llama **overfitting** (sobreajuste) y es enemigo número uno del Machine Learning.

11.2 La solución: dividir los datos

Partimos los datos en dos grupos antes de entrenar nada:

- **Train (entrenamiento):** el modelo los usa para aprender. Suele ser el 70-80% de los datos.
- **Test (prueba):** el modelo no los ve durante el entrenamiento. Después los usamos para evaluar de forma honesta cómo predice en datos que nunca ha visto.

Si el modelo funciona bien en test (no solo en train), sabemos que ha aprendido patrones reales y generaliza.

11.3 En sklearn

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,        # 20% a test, 80% a train
    random_state=42        # semilla aleatoria para que la división sea re-
producible
)
```

Qué hace cada parámetro:

- `test_size=0.2`: proporción de datos que van a test. Valores típicos: 0.2 o 0.3.

- `random_state=42`: fija la semilla del generador aleatorio. Si lo quitas, cada vez que ejecutes saldrá una división distinta; si lo fijas, siempre sale la misma. Esto es útil para reproducir resultados.

> **REGLA INQUEBRANTABLE**
>
> Evalúa **SIEMPRE** en test, nunca en train. Las métricas en train son demasiado optimistas porque el modelo ya vio esos datos. La métrica que realmente importa es la de test.

11.4 Splits alternativos: por posición

A veces el enunciado pide un split distinto. Por ejemplo, «los últimos 20 registros a test». Eso se hace con slicing, no con train_test_split:

```
X_train = X[:-20]   # todo excepto los últimos 20
X_test  = X[-20:]   # los 20 últimos
y_train = y[:-20]
y_test  = y[-20:]
```

Este tipo de split tiene sentido en series temporales (donde no puedes usar el futuro para predecir el pasado), pero en problemas generales suele ser preferible el split aleatorio.

12. Overfitting, Underfitting y el dilema sesgo-varianza

Esta sección es uno de los conceptos más importantes de todo el bloque de ML. Entenderlo bien te va a ahorrar muchísimas horas de frustración.

12.1 Las dos formas de fracasar de un modelo

> **LAS DOS ENFERMEDADES POSIBLES**
>
> **Underfitting (subajuste):** el modelo es demasiado simple para capturar la complejidad de los datos. Falla tanto en train como en test. Como intentar trazar una recta para datos que claramente siguen una curva.
>
> **Overfitting (sobreajuste):** el modelo es demasiado complejo y se ha aprendido los datos de memoria, incluido el ruido aleatorio. Acierta casi todo en train, pero falla en test. Como un alumno que se sabe los exámenes de memoria pero no ha entendido la materia.

12.2 El gráfico clave: error vs complejidad

Hay un gráfico que aparece en toda la bibliografía de ML y que conviene tener mentalmente grabado:

- En el eje X está la **complejidad del modelo** (grado del polinomio, número de features, profundidad del árbol…).

- En el eje Y está el **error**.
- Se dibujan dos curvas: el error en **train** (azul) y el error en **test/validación** (verde).

La forma típica es:

- **Con poca complejidad** (izquierda del gráfico): ambos errores son altos. Estamos en zona de *underfitting*. El modelo es demasiado simple.
- **Con complejidad media**: ambos errores son bajos y parecidos. **Esta es la zona óptima.**
- **Con mucha complejidad** (derecha): el error de train sigue bajando pero el de test empieza a subir. Estamos en zona de *overfitting*.

> **CÓMO DETECTAR OVERFITTING SIN EL GRÁFICO**
>
> Si al comparar métricas obtienes: *R^2 train = 0.95 y R^2 test = 0.60* (gran diferencia), estás en overfitting seguro. El modelo se sabe los datos de train de memoria pero no generaliza.
>
> Si obtienes: *R^2 train = 0.55 y R^2 test = 0.52* (parecidos, ambos mediocres), el problema no es overfitting sino underfitting: el modelo es demasiado simple.

12.3 El dilema sesgo-varianza (bias-variance tradeoff)

Los dos tipos de error se explican con dos conceptos estadísticos:

- **Sesgo (bias):** error por ser demasiado simple. Un modelo con alto sesgo hace suposiciones muy rígidas que no corresponden a la realidad. Underfitting = alto sesgo.
- **Varianza (variance):** error por ser demasiado sensible a los datos concretos de entrenamiento. Un modelo con alta varianza cambia mucho si cambias un poco los datos. Overfitting = alta varianza.

El dilema es que normalmente **reducir uno aumenta el otro**. Un modelo muy simple tiene mucho sesgo y poca varianza; uno muy complejo, poca sesgo pero mucha varianza. El arte es encontrar el punto óptimo.

La regularización (Ridge, Lasso, Elastic Net que veremos en la Parte V) son precisamente técnicas para controlar este equilibrio.

13. StandardScaler y MinMaxScaler

13.1 Por qué escalar

Imagínate que tienes dos features con escalas muy distintas: edad (entre 20 y 50) e ingresos anuales (entre 20.000 y 100.000). Algunos algoritmos se ven afectados por esas escalas. Además, los coeficientes β de la regresión salen en las unidades de cada variable, lo que hace imposible compararlos.

13.2 StandardScaler

Transforma cada feature para que tenga media 0 y desviación típica 1:

$$x_{escalada} = (x - media) \ / \ desviación_típica$$

EJEMPLO

Si la edad media en tu grupo es 32 y la desviación típica 6:

- Alguien de 38 años → (38 − 32)/6 = +1 (una desviación por encima de la media)
- Alguien de 26 años → (26 − 32)/6 = −1
- Alguien de 32 años → 0 (justo en la media)

Ahora la edad está en desviaciones típicas, no en años. Si haces lo mismo con los ingresos, las dos variables están en la misma escala.

13.3 MinMaxScaler

Alternativa al StandardScaler. Transforma cada feature para que sus valores estén entre 0 y 1:

$$x_{escalada} = (x - x_{min}) \ / \ (x_{max} - x_{min})$$

El valor mínimo queda en 0, el máximo en 1, el resto proporcionalmente entre medias.

Cuándo usar qué	Preferible
La distribución es aproximadamente normal	StandardScaler
La distribución no es normal o hay outliers grandes	MinMaxScaler (el StandardScaler se ve más afectado por outliers)
Necesito valores estrictamente en [0,1] (por ejemplo, para redes neuronales)	MinMaxScaler
En duda	StandardScaler por defecto

13.4 El peligro: contaminación train/test

ERROR GRAVÍSIMO Y MUY FRECUENTE

Si aplicas el scaler a *todo el dataset antes de hacer el split*, estás mezclando información de test con train: el modelo «ve» indirectamente estadísticas del test. Tus métricas quedarán infladas y engañosas.

La forma correcta es ajustar el scaler **solo con los datos de train**, y después aplicar la misma transformación a test.

En el notebook de Advertising de tu clase hay un ejemplo muy clarificador:

CASO REAL

Tenemos edades: [20, 35, 40, 25, 18]

Train: [20, 35, 40, 25]. Test: [18].

Incorrecto (scaler sobre todo X):

- Mínimo = 18, máximo = 40
- 18 se escala a 0 (el mínimo)

Correcto (scaler solo sobre train):

- Mínimo train = 20, máximo train = 40
- 18 (en test) se escala a −0.1 — fuera del rango [0,1]

En el mundo real, los datos de producción pueden tener valores fuera del rango de train. Lo correcto es que el scaler se ajuste solo a train y admita que test pueda quedar fuera de [0,1].

13.5 Forma correcta en sklearn

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
scaler.fit(X_train)                  # aprende media y desviación SOLO de
train
X_train_scaled = scaler.transform(X_train)
X_test_scaled  = scaler.transform(X_test)   # aplica la misma transformación
a test
```

También existe el atajo `fit_transform` que hace fit y transform en una sola línea:

```
X_train_scaled = scaler.fit_transform(X_train)   # solo con train
X_test_scaled  = scaler.transform(X_test)        # NUNCA fit_transform con
test
```

REGLA MENTAL

fit solo con train. *transform* con train y con test por separado. **Nunca** hagas fit con test.

13.6 ¿Escalar mejora la regresión lineal?

En la regresión lineal *sin regularización*, escalar **NO cambia las predicciones ni las métricas de error**. Solo cambia la escala de los coeficientes. Entonces, ¿por qué se hace?

1. **Para comparar la importancia de las features.** Con los datos escalados, cada coeficiente mide el efecto de 1 desviación típica, así que son comparables entre sí.
2. **Para los modelos con regularización (Ridge, Lasso, Elastic Net).** Ahí el escalado es casi obligatorio porque la penalización depende de la magnitud de los coeficientes.
3. **Para los modelos basados en Gradient Descent.** Si las escalas son muy distintas, el gradiente no apunta eficientemente al mínimo y tarda mucho en converger.

14. Validación cruzada (cross_val_score)

14.1 La limitación del train/test simple

Con un solo split train/test, los resultados dependen de qué muestras cayeron en cada grupo. Si tuviste suerte y las 20 muestras de test son fáciles, tus métricas saldrán brillantes. Si fueron difíciles, malas. No puedes fiarte del todo de un solo split.

14.2 La idea: dividir en K pliegues

La **validación cruzada** (cross-validation, CV) consiste en dividir los datos en K partes iguales (típicamente K = 5 o 10). Luego:

1. Entreno con 9 partes y evalúo con 1.
2. Repito cambiando qué parte queda fuera.
3. Al final tengo 10 métricas distintas, una por cada iteración.
4. Calculo la media y la desviación típica de esas 10 métricas.

La media te da una estimación más robusta del rendimiento del modelo. La desviación típica te dice lo variable que es según con qué datos entrenes: si es pequeña, tu modelo es estable; si es grande, tu modelo es sensible a los datos.

14.3 En sklearn

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(modelo, X, y,
                         cv=10,                          # 10 pliegues
                         scoring="neg_mean_absolute_error")

print("Scores por pliegue:", scores)
print("Media:", scores.mean())
print("Desviación típica:", scores.std())
```

¿POR QUÉ "NEG_MEAN_ABSOLUTE_ERROR" (NEGATIVO)?

Sklearn tiene la convención de que «más alto = mejor» para todos los scores. Pero el MAE es al revés: más bajo es mejor. Para resolverlo, sklearn le pone un signo negativo. Un MAE de 2.5 se reporta como −2.5. Tienes que tenerlo en cuenta al leer los resultados.

15. Condiciones para aplicar regresión lineal

La regresión lineal hace suposiciones sobre los datos. Si se violan gravemente, el modelo puede dar resultados engañosos (métricas buenas pero inferencia incorrecta). Estas condiciones son las que viste (o iba a ver) en las últimas diapositivas de la presentación de clase.

15.1 Linealidad entre target y predictores

La relación entre cada X y Y debe ser aproximadamente lineal. Se revisa con la matriz de correlaciones y scatter plots. Si los puntos forman una curva clara (una U, por ejemplo), la regresión lineal no va a funcionar bien y deberías plantearte una regresión polinómica (Parte IV).

15.2 No multicolinealidad

Los predictores deben ser razonablemente independientes entre sí. Si dos features correlacionan mucho (por ejemplo, $r > 0.8$), los coeficientes del modelo se vuelven inestables y difíciles de interpretar.

POR QUÉ LA MULTICOLINEALIDAD ES UN PROBLEMA

Imagina que tienes dos features casi idénticas (altura en metros y altura en centímetros). El modelo no sabe si el efecto sobre Y viene de una o de otra. Puede asignar coeficientes enormes y opuestos que se cancelan entre sí, volviendo el modelo inestable y sin sentido interpretativo.

Se detecta con la matriz de correlaciones entre features. Si dos features tienen correlación alta, se suele eliminar una de las dos (o combinarlas).

15.3 Homocedasticidad

Este palabro viene del griego y significa «igual varianza». La idea es que la **dispersión de los errores debe ser constante a lo largo del rango de las predicciones**.

- **Distribución homocedástica** (buena): los residuos tienen una dispersión parecida tanto para predicciones pequeñas como grandes.
- **Distribución heterocedástica** (mala): la dispersión cambia con la magnitud de la predicción. Por ejemplo, el modelo predice bien los precios bajos pero muy mal los altos.

Se detecta con un scatter plot de residuos vs predicciones. Si ves un patrón de «embudo» (más disperso a la derecha), tienes heterocedasticidad.

15.4 Independencia y normalidad de los residuos

- **Independencia:** los residuos no deben estar correlacionados entre sí. Si lo están (por ejemplo, en series temporales donde el error de un mes depende del anterior), la regresión lineal clásica no es apropiada.
- **Normalidad:** los residuos deben seguir aproximadamente una distribución normal. Se comprueba con un histograma: debe tener forma de campana centrada en 0.

15.5 Normalidad del target

No es estrictamente obligatorio, pero si el target tiene una distribución muy sesgada (por ejemplo, salarios con una cola larga a la derecha), el modelo puede funcionar peor. A veces conviene aplicar una transformación logarítmica al target antes de entrenar.

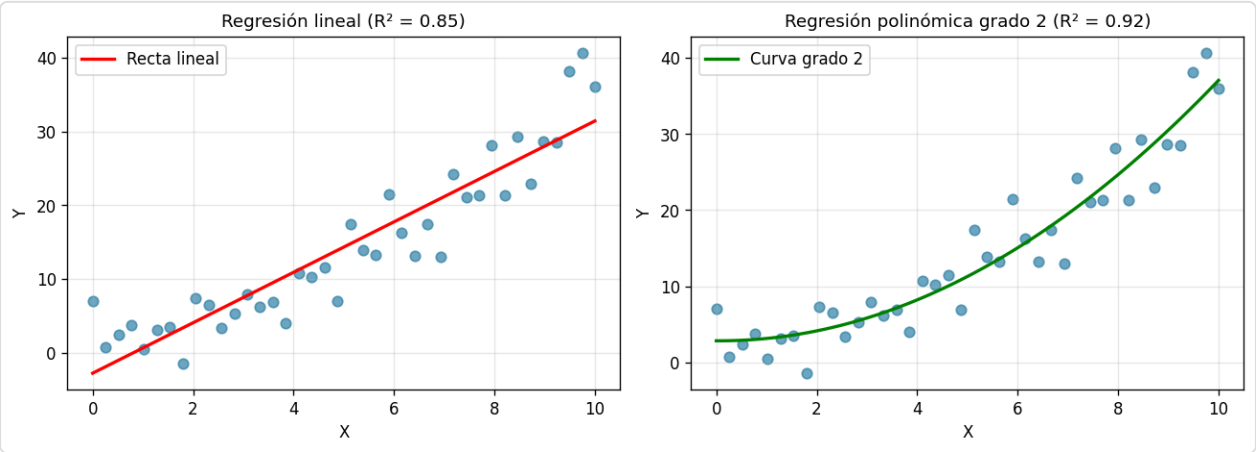
> **EN LA PRÁCTICA**
>
> En un análisis exploratorio típico para regresión lineal, siempre:
>
> 1. Dibujas el histograma del target para ver su distribución.
> 2. Dibujas la matriz de correlaciones con heatmap.
> 3. Dibujas scatter plots entre target y cada feature importante.
> 4. Después de entrenar, dibujas el histograma de residuos para comprobar normalidad.

Parte IV
Regresión polinómica

Hasta ahora hemos supuesto que la relación entre X e Y es una línea recta. Pero en la vida real muchas relaciones son curvas. La regresión polinómica es una extensión muy elegante de la regresión lineal que nos permite capturar esas curvas sin salirnos del mundo lineal.

16. Cuando la relación no es lineal

Supón que pintas una nube de puntos y ves claramente que siguen una forma curva: primero suben despacio, después más rápido, etc. Si intentas ajustar una recta, el modelo va a fallar sistemáticamente en las zonas curvas. La mejor recta posible aún será un mal modelo.



A la izquierda, una recta intentando ajustar puntos que siguen una curva: ajuste pobre (R^2 bajo). A la derecha, la misma nube con una curva polinómica de grado 2: se adapta a la forma real de los datos.

La idea genial de la regresión polinómica es: **añadir columnas nuevas al dataset que sean potencias de las originales**. Si X es la edad, creamos X^2, X^3, X^4... y las tratamos como si fueran features independientes. Después, aplicamos una regresión lineal clásica sobre ese dataset ampliado.

La ecuación que obtendremos tiene forma:

$$Y = \beta_0 + \beta_1 \cdot X + \beta_2 \cdot X^2 + \beta_3 \cdot X^3 + \ldots$$

POR QUÉ LA REGRESIÓN POLINÓMICA SIGUE SIENDO "LINEAL"

Aunque la curva resultante sea curvada, los parámetros β siguen apareciendo de forma lineal en la ecuación (multiplicados, no elevados). Por eso técnicamente sigue siendo una regresión *lineal* (lineal en los parámetros). Lo que cambiamos son las features, no el método de ajuste. Esto significa que podemos usar la misma `LinearRegression` de sklearn para entrenarla.

17. PolynomialFeatures de sklearn

Sklearn tiene una herramienta llamada **PolynomialFeatures** que automáticamente genera todas las potencias y combinaciones de las features originales hasta el grado que le pidas.

PolynomialFeatures(degree=3) crea columnas nuevas: x, x^2, x^3

x	x^2	x^3
2	4	8
3	9	27
5	25	125

PolynomialFeatures transforma cada valor de X en varias columnas: el propio X, X^2, X^3...

Si tienes **una sola feature** X y usas `PolynomialFeatures(degree=3)`, obtienes 3 columnas nuevas: X, X^2, X^3. Con una segunda feature Y añadida y degree=2, obtienes: X, Y, X^2, X·Y, Y^2. Es decir, no solo las potencias, también los **términos cruzados** (productos entre features). Esto es muy potente porque captura interacciones entre variables.

17.1 Uso típico en sklearn

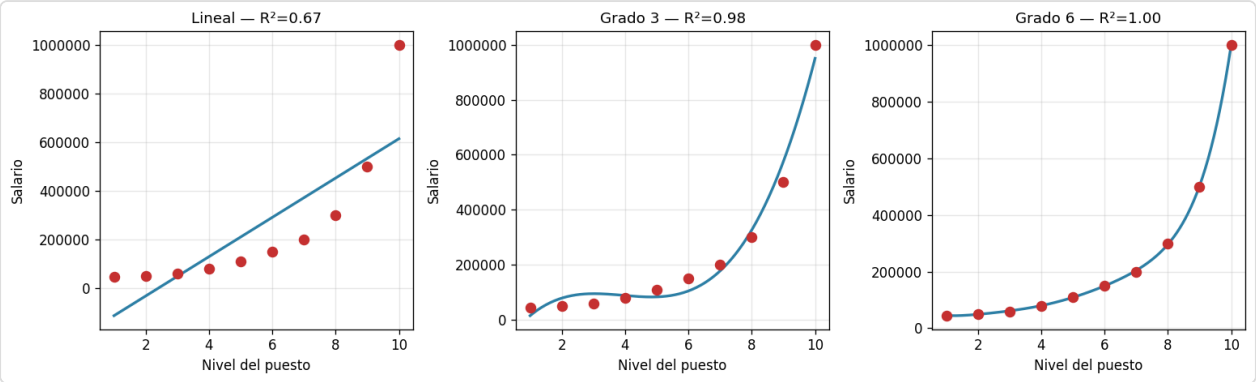
```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

# Paso 1: crear el generador de features polinómicas
poly_feats = PolynomialFeatures(degree=2)

# Paso 2: ajustar con los datos de entrenamiento (aprende las columnas a
crear)
poly_feats.fit(X_train)

# Paso 3: transformar train y test en sus versiones polinómicas
X_train_poly = poly_feats.transform(X_train)
X_test_poly  = poly_feats.transform(X_test)

# Paso 4: entrenar una regresión lineal clásica sobre los datos ampliados
modelo = LinearRegression()
modelo.fit(X_train_poly, y_train)

# Paso 5: predecir (no olvides transformar también los datos nuevos)
y_pred = modelo.predict(X_test_poly)
```

ERROR TÍPICO: OLVIDAR TRANSFORMAR EL TEST

Una vez has entrenado con `X_train_poly`, el modelo espera features polinómicas siempre. Si luego le pasas `X_test` sin transformar, te dará un error de dimensiones o predicciones absurdas. Regla: si entrené con polinómica, predigo con polinómica.

17.2 El ejemplo del notebook de clase: salarios por nivel de puesto

En uno de los notebooks que vimos, el dataset *position_salaries* tenía una relación claramente exponencial: los puestos altos cobran *muchísimo* más que los bajos, no solo un poco más. La regresión lineal no podía capturar esa aceleración.



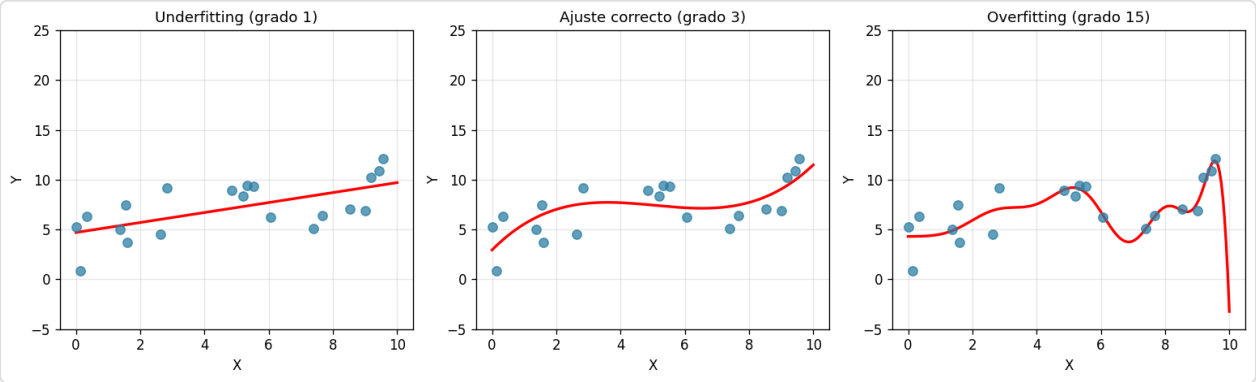
Dataset real de salarios por nivel. La regresión lineal (izquierda) falla porque la relación no es lineal. Grado 3 captura la forma general. Grado 6 se ajusta casi a la perfección.

Fíjate que el R^2 en train mejora mucho pasando de grado 1 a grado 6. Pero ojo: **mejorar el R^2 en train no siempre es bueno**. Aquí entra la advertencia del dilema sesgo-varianza que vimos en la Parte III.

18. Elección del grado óptimo

La pregunta natural es: ¿cuanto más alto el grado, mejor el modelo? **No.** Llegado un punto, el modelo se vuelve tan flexible que empieza a memorizar los datos de train, incluido el ruido. Eso es *overfitting*.

18.1 La imagen que tienes que grabarte

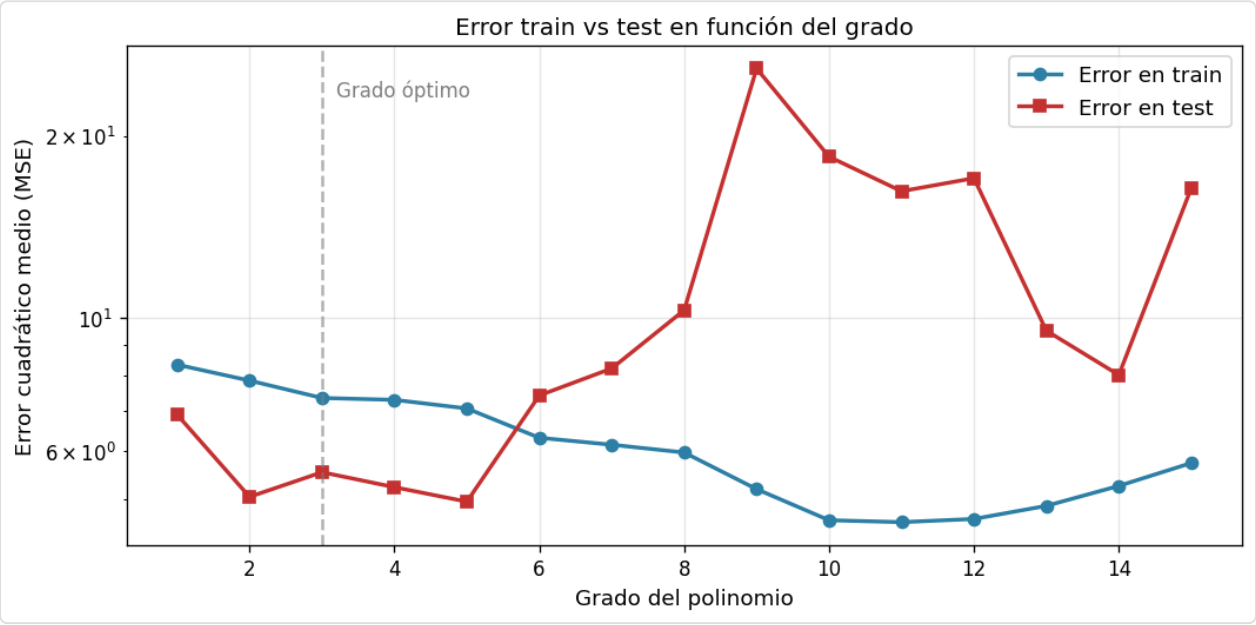


Tres modelos sobre los mismos datos. Grado 1 (izquierda): demasiado rígido, no captura la tendencia. Grado 3 (centro): sigue la forma general sin obsesionarse con cada punto. Grado 15 (derecha): la curva hace piruetas para pasar cerca de cada punto, incluida la parte del ruido aleatorio. Esto último es overfitting en estado puro.

Visualmente el grado 15 parece "ajustarse mejor" porque la curva está más cerca de cada punto. Pero fíjate en la parte derecha, cómo la curva cae en picado hacia abajo: eso es pura invención del modelo. Si apareciera un punto nuevo en X=10.5, grado 15 predeciría algo absurdo.

18.2 El gráfico diagnóstico: error en train vs error en test

La única forma honesta de elegir el grado es comparar el error en train con el error en test para distintos grados:



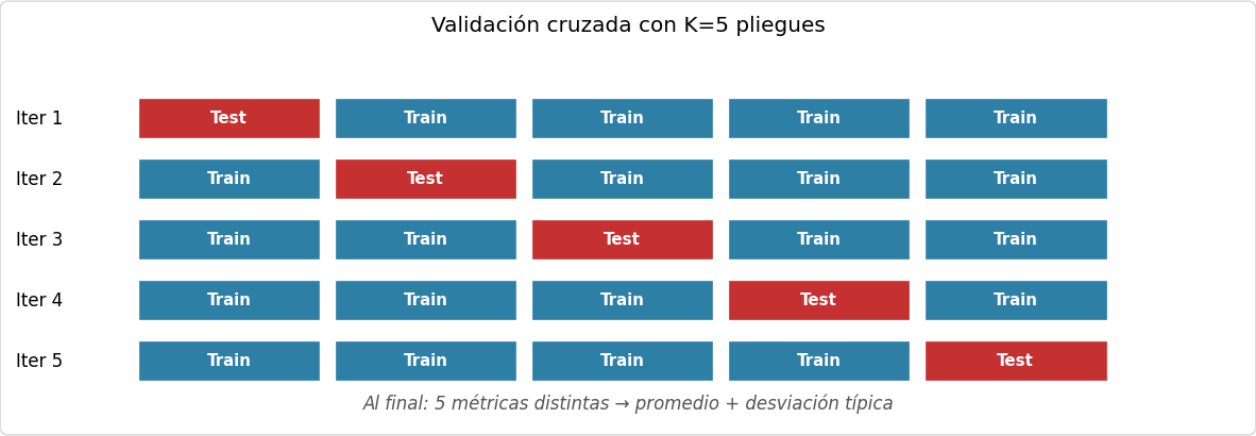
El error de train (azul) sigue bajando conforme el grado aumenta: el modelo cada vez se ajusta mejor a los datos que ha visto. Pero el error de test (rojo) baja al principio, toca un mínimo y después empieza a dispararse. Ese punto en el que el test toca su mínimo es el grado óptimo.

LA REGLA DE ORO

El grado óptimo es aquel que **minimiza el error en test**, no en train. Si el error en train es muy bajo pero el de test es alto, tienes overfitting: baja el grado. Si los dos errores son altos, tienes underfitting: sube el grado o añade más features.

18.3 Validación cruzada para elegir el grado

Con un solo split train/test la evaluación es ruidosa: depende de qué muestras cayeron en test. Para elegir el grado con más fiabilidad, usamos **validación cruzada** (que ya vimos en la Parte III). La idea recordatoria:



Con K=5 pliegues, entrenamos el modelo 5 veces. Cada vez, un pliegue distinto hace de test y los otros 4 de train. Al final tenemos 5 métricas: calculamos la media (estimación del rendimiento) y la desviación típica (estabilidad).

Aplicado al problema de elegir el grado:

```
from sklearn.model_selection import cross_val_score

for grado in [1, 2, 3, 4, 5, 6]:
    pf = PolynomialFeatures(degree=grado)
    X_poly = pf.fit_transform(X)
    scores = cross_val_score(LinearRegression(), X_poly, y,
                             cv=10, scoring="neg_mean_absolute_error")
    print(f"Grado {grado}: MAE medio = {-scores.mean():.2f}, std =
{scores.std():.2f}")
```

Elegimos el grado que dé mejor MAE medio con desviación típica razonable. Si dos grados empatan en MAE medio, elige el de menor desviación típica (más estable) y el menor grado posible (principio de parsimonia: a igualdad de rendimiento, modelos simples son preferibles).

18.4 Resumen operacional de regresión polinómica

Paso	Qué hacer
1	Visualizar X vs Y con scatter plot para detectar si la relación es curva
2	Partir los datos en train/test antes de nada
3	Probar varios grados (1, 2, 3, 4, 5, 6) y comparar error train vs test
4	Elegir el grado que minimice error en test
5	(Recomendado) Confirmar la elección con validación cruzada
6	Cuidado con grados muy altos: el overfitting explota rápido

PARA RETENER

La regresión polinómica **no es un modelo distinto** de la lineal. Es la misma regresión lineal aplicada a features transformadas. Todo lo que aprendimos sobre coeficientes, R^2, MAE, train/test se aplica exactamente igual. El único hiperparámetro nuevo es el *grado*, y se elige con el mismo diagnóstico que cualquier hiperparámetro: probando varios valores y comparando el rendimiento en test.